

Computational Social Science

Week 9: Using GitHub

Bariş Arı

20 April 2026

Last week we installed Git, configured our identity, and learned the local workflow: `git init`, `git add`, `git commit`, `git log`, and `git checkout`.

This week we move from our local computer environment to the web. We will start by **downloading** an existing project from GitHub, then learn how to **upload** our own work, keep it in sync with the online copy, and propose changes using **branches** and **pull requests**.

As usual, there is some terminology involved.

- Downloading a project is called **cloning**.
- The online copy of a repository is called a **remote**.
- Downloading updates from a remote repository is called **pulling**.
- Uploading your commits to the remote is called **pushing**.

These terms are unfamiliar at the moment, but they will feel natural as we practice.

1 Before We Start: Quick Recap

You should already be able to do the following on your own machine:

1. Create a directory and place some files inside it.
2. Turn that folder into a Git repository with `git init`.
3. Stage changes with `git add` and record them with `git commit -m "message"`.
4. Inspect the history with `git log` and the current state with `git status`.

If any of these are still fuzzy, revisit the Week 8 handout before continuing. For this session you also need a **GitHub account** (homework from Week 8).

2 Downloading a Project: Cloning

When you find a project on GitHub that you want to work with — a paper’s replication files, a course repository, or a colleague’s code — you need to download it to your computer. In Git, this is called **cloning**.

Cloning does two things at once: it downloads all the project files, and it also downloads the full history of every change ever made to those files. You get not just the current version, but the entire record of how the project developed.

You do not need a GitHub account to clone a public repository. It is a straightforward download.

How to Clone a Repository

1. Go to the repository page on GitHub.
2. Click the green **Code** button.
3. Make sure **HTTPS** is selected, and copy the URL shown.
4. Open a terminal and navigate to the folder where you want the project to live.
5. Run:

```
git clone https://github.com/USERNAME/REPOSITORY.git
```

Git will create a new folder with the repository name inside your current directory. All the project files will be inside it.

To download the practice repository:

```
git clone https://github.com/LadyKerr/GitGoing.git
```

This creates a folder called **GitGoing** on your computer. You can open it like any other folder.

Using the commands we used last week, check its status and versions.

If you wish, you can try another practice repository:

```
git clone https://github.com/octocat/Hello-World.git
```

i Note

You only clone once. Once the folder is on your machine, you do not need to clone it again. To get updates later, see the next section.

💡 Tip

If you have already cloned a repository, you do *not* need to run `git init` inside it. Cloning already creates the `.git` folder for you.

3 Updating: Pulling Changes from GitHub

Once a project lives on your laptop, the online version on GitHub can still change: a collaborator may push updates, or you may edit a file directly in the GitHub web interface. To bring those changes down to your computer, run `git pull` *inside the repository folder*:

```
git pull
```

A `git pull` does two things in sequence:

1. `git fetch`: downloads the latest information about the remote.
2. `git merge`: integrates those changes into your current branch.

Habit to develop: before you start a new work session, `git pull` first. This prevents most conflicts between your local version and the one on GitHub.

4 Signing in to GitHub from Your Computer

So far you have only *read* from GitHub: cloning a public repository and pulling updates does not require any sign-in. That changes the moment you want to **write** to GitHub: uploading your own project, pushing changes, or cloning a private repository.

When you ask your computer to write to GitHub, GitHub needs to know *who* is making the request. Your GitHub web password is **not** accepted on the command line. Instead, you authenticate using one of two standard methods:

- **Method A: HTTPS with a Personal Access Token (PAT).** Easier to set up, recommended for beginners.
- **Method B: SSH keys.** A little more setup, but no expiring tokens to manage later on.

You only need **one** of these to work. We will cover both so you can pick what suits you.

Method A: HTTPS with a Personal Access Token (PAT)

A Personal Access Token (PAT) is a long string that acts as a password for your command line.

When you push, Git connects over the standard web protocol (HTTPS, the same one your browser uses). GitHub then asks: who are you? You prove your identity with a Personal

Access Token — a long auto-generated string you copy from your GitHub settings. It acts like a password, but it is scoped (you choose what it can do) and has an expiry date.

The downside: when the token expires, you generate a new one and update it on your machine. Most operating systems have a credential manager that stores it so you are not pasting it every time, but it still needs refreshing periodically.

How to use it:

1. Sign in to github.com in your browser.
2. Click your profile picture (top right) → **Settings**.
3. In the left sidebar, scroll to **Developer settings**.
4. Open **Personal access tokens** → **Tokens (classic)** → **Generate new token (classic)**.
5. Give the token a name (for example `my css project`), set an expiration date, and tick the **repo** scope. Selecting **repo** gives the token permission to read and write your repositories. This is needed for pushing changes.
6. Click **Generate token** and **copy the token immediately**. GitHub will not show it again.
7. The first time you push to GitHub, Git will ask for your **username** and **password**. Use your GitHub username and paste the **token** as the password.

On Windows, Git Credential Manager will remember it after the first time. On Mac, the system Keychain stores it automatically. On Linux it needs a one-line setup (`git config --global credential.helper store`).

Warning

Treat your token like a password. Do not paste it into documents, chat messages, or commit it into a repository. Anyone with the token can act on your GitHub account.

Method B: SSH Keys

SSH stands for Secure Shell. It works differently at a technical level. Instead of a password-like token, your computer generates a matched pair of keys: a private key (stays on your machine, never shared) and a public key (you paste it into your GitHub settings once). When you push, GitHub challenges your computer to prove it holds the private key. Your computer uses the private key to prove its identity silently.

In short, with SSH, your computer proves its identity to GitHub using a cryptographic key pair.

Once set up, it usually works well: no expiry, no tokens to renew, no credential manager needed. However, it is occasionally blocked on restricted computers and networks (University computers).

How to use it:

1. **Generate a key pair.**

```
ssh-keygen -t ed25519 -C "your@email.com"
```

Press **Enter** to accept the defaults. You will be asked for a passphrase; you can press **Enter** twice to skip it.

This creates two files in `~/.ssh/`: `id_ed25519` (the private key) and `id_ed25519.pub` (the public key).

2. **Print the public key so you can copy it:**

```
cat ~/.ssh/id_ed25519.pub
```

Select the whole line of output, starting with `ssh-ed25519` and ending with your email, and copy it.

3. **Add the key to GitHub.** In your browser, go to GitHub → **Settings** → **SSH and GPG keys** → **New SSH key**. Give it a title that identifies your machine (e.g. *my laptop*), paste the key into the **Key** field, and click **Add SSH key**.

4. **Test the connection:**

```
ssh -T git@github.com
```

The first time, you will be asked to confirm GitHub’s fingerprint: type **yes** and press **Enter**. You should then see:

```
Hi username! You've successfully authenticated,  
but GitHub does not provide shell access.
```

The “does not provide shell access” part is expected. It means the key works.

 Warning

The file `id_ed25519` (without `.pub`) is your **private key**. Never share it, never upload it anywhere. Only the `.pub` file goes onto GitHub.

5 Creating a Repository on GitHub

A GitHub repository (often shortened to **repo**) is the online home of a project. It is where your code lives on GitHub, together with its full history, any collaborators, and project-management tools such as issues and pull requests.

In this section we create an empty repository on GitHub. In the next section, we connect our existing Week 8 folder to it and upload our work.

5.1 Create the repository

- In your browser, go to <https://github.com> and click the + icon (top right) → **New repository**.
- Give the repository a name, for example *learning_git*. The name should be short, lowercase, and without spaces. Use underscores or hyphens if you need a separator.
- Add a short description (optional). This appears at the top of the repository page and helps others (and your future self) understand what the project is about.
- Choose **Public** (anyone on the internet can see it) or **Private** (only you and people you explicitly invite). You can change this later in the repository settings.
- For now, **do not** tick *Add a README file*, *.gitignore*, or *Choose a license*. We will push our existing Week 8 folder into this repo, and ticking these options would create conflicting files on GitHub before we upload anything.
- Click **Create repository**.

GitHub then shows a page titled something like *Quick setup — if you've done this kind of thing before*. Keep this tab open: it displays the repository URL we need in the next section.

Tip

The URL shown on the setup page has two forms: an **HTTPS** URL (starting with `https://github.com/...`) and an **SSH** URL (starting with `git@github.com:...`). Which one you use depends on the authentication method you use. If you chose a PAT, use HTTPS. If you chose SSH keys, use SSH.

Note

An empty repository is a repository with no commits yet. GitHub treats this differently from a repository with content — the setup page will look different once you push your first commit, and some features (like branch protection) only become available after the first commit lands.

6 Connecting Your Local Repository to GitHub

Your *learning_git* folder from Week 8 already has a full Git history on your machine. We now tell it where its online counterpart lives. The online counterpart is called a **remote**, and by convention its name is **origin**.

Open a terminal inside your *learning_git* folder and run:

```
git remote add origin https://github.com/YOUR-USERNAME/learning_git.git
```

Replace **YOUR-USERNAME** with your GitHub username. If you chose SSH in the previous section, use the SSH URL instead:

```
git remote add origin git@github.com:YOUR-USERNAME/learning_git.git
```

Check that the remote was recorded:

```
git remote -v
```

You should see `origin` listed twice: once for fetching, once for pushing.

Note

Connecting is not uploading. The `git remote add` command only records the remote's address in your local Git configuration: it is like saving a contact in your phone. Nothing has been sent to GitHub yet, and if you refresh your repository page in the browser, it will still be empty. The actual upload happens later, when we run `git push`.

Tip

Older versions of Git called the default branch `master`. GitHub now uses `main`. If `git status` shows you are on `master`, rename the branch once with `git branch -M main`.

7 Pushing Your Work to GitHub

Pushing uploads your local commits to the remote. The first push also tells Git which remote branch to track by default.

But before we push anything, we need to make sure we are not uploading anything we should not. Pushing is effectively permanent: once a file lands on GitHub, removing it from future commits is not enough, because the full history still contains it. You may need to delete the whole repository and start over. Let's avoid that by setting up a safeguard first.

7.1 Telling Git What to Ignore: `.gitignore`

Most projects contain files you do *not* want to share: credentials, large datasets, cached output, or your operating system's metadata. A special file called `.gitignore` lists patterns that Git should pretend do not exist.

Create a file called `.gitignore` in the root of your project and add lines to specify what to ignore. You can create it from the terminal or from VS Code: open the *learning_git* folder, click the **New File** icon in the Explorer panel, and name the file `.gitignore` (including the leading dot). Add the names of the files you want to Git to ignore, such as `my_deepest_secret.txt`.

So far we have told Git to ignore one file. In practice, every project accumulates a handful of files that should never be shared —not because they are secret, but because they are irrelevant, machine-specific, or too large. It is common practice to set these up right at the start of a project. Here is a more complete `.gitignore` you can use as a starting point:

```
# Operating system files
.DS_Store
Thumbs.db

# Python
__pycache__/
*.pyc
.venv/

# R / RStudio
.Rhistory
.Rproj.user/

# Secrets
.env
credentials.json
my_deepest_secret.txt
secret_*. * # use of wildcard

# Local data (too large or confidential)
data/raw/
```

Each block handles a different kind of clutter:

- **Operating system files.** macOS silently creates `.DS_Store` files in every folder you open in Finder; Windows does the same with `Thumbs.db`. They are invisible to you but show up in `git status` and would otherwise end up on GitHub.
- **Secrets and private files.** As discussed above. Anything containing passwords, API keys, tokens, or personal data belongs here.
- **Python.** When Python runs a script, it generates compiled files (`.pyc`) and a `__pycache__` folder for speed. These are regenerated automatically and should not be shared. `.venv` is a virtual environment folder — tens of megabytes of dependencies that belong on each machine separately, not in the repository.
- **R / RStudio.** `.Rhistory` logs every command you have typed in the R console, and `.Rproj.user` stores your personal RStudio settings. Neither is useful to anyone else.
- **Local data.** Raw datasets are often too large for GitHub (there is a 100 MB file

limit) or confidential (survey responses, unpublished data). Keep them on your machine and share them through other means.

- You do not have to list every file by name. `.gitignore` supports the `*` wildcard, which matches any sequence of characters. This is useful when you have a naming convention for private files. For example, we told Git to ignore any file whose name starts with `secret_`.

Tip

Adopting a naming convention for private files (e.g. starting every one with `secret_` or placing them in a `private/` folder) means you can protect them with a single line in `.gitignore`, and you are far less likely to accidentally commit one.

7.1.1 Committing `.gitignore` itself

Do not forget to commit the `.gitignore` itself:

```
git add .gitignore
git commit -m "Add .gitignore"
git push
```

Warning

`.gitignore` only prevents *new* files from being tracked. If you have already committed a sensitive file in the past, rewriting history is required to remove it. Always add files to `.gitignore` *before* the first commit.

Warning

`.gitignore` hides *files*, not *content* of files that are in not ignored. If you hard-code a password, API key, or personal data directly into a script that **is** tracked (say, `analysis.py` or `report.md`), `.gitignore` will not help you — the secret gets pushed along with the code.

The habit to build: keep secrets in a separate file (e.g. `.env` or `credentials.json`), list that file in `.gitignore`, and have your code read from it. Never paste credentials into the script itself.

7.2 Now We Are Ready to Push

```
git push -u origin main
```

The `-u` flag (short for `--set-upstream`) only needs to be used once. After that, `git push` with no arguments will push the current branch to `origin`.

Refresh your GitHub repository page in the browser. You should see *first_file.txt* and your earlier commit messages. Congratulations! Your work is now backed up online.

From now on, the routine is simply:

```
git add .  
git commit -m "Describe the change"  
git push
```